Application Architecture & Technology Stack

The **Cherry Precision App** is a modern, cross-platform desktop application built for high-performance AirGauge monitoring, offline USB reading ingestion, and historical data analytics. Instead of traditional monolithic desktop frameworks, this application leverages the lightweight, secure **Tauri framework** to blend web frontend technologies with a systems programming backend.

1. The Frontend (UI & Interactions)

The user interface is built securely without heavyweight JavaScript frameworks to ensure lightning-fast startup times, zero bloat, and long-term maintainability.

- **Core Technologies:** Vanilla HTML5, CSS3, and JavaScript (ES6+).
- **Styling:** Custom modular CSS. Designed around a sleek dark theme (`index.css`) emphasizing rapid visual feedback, flexible grid layouts (Flexbox/Grid), and readable data matrices logic natively.
- **Visualizations: Chart.js** is integrated to handle all complex analytics components, powering the X-Bar and Range control charts natively within the Statistical Process Control (SPC) dashboard.
- **Interactions:** Single-page architecture logic handled purely via Vanilla JavaScript DOM manipulation (`main.js`), avoiding overhead from Virtual DOMs.

2. The Backend (Core Logic & Native OS Integrity)

The backend requires low-level system access to communicate seamlessly with hardware via serial COM ports and to parse bulky database archives. This is why **Rust** was chosen.

- **Core Framework: Tauri (Rust)**. Tauri acts as the protective bridge, allowing the frontend to render elegantly via the OS's native webview component while the Rust backend handles intensive processing safely in the background.
- **Hardware Interface:** The Rust backend natively communicates with COM ports utilizing robust internal serial packages, establishing the critical handshake protocol (`*...#` raw datastream interpretation) from your AirGauge systems.
- **Language Profile:** Rust provides unparalleled memory safety without a garbage collector, ensuring that serial monitoring can run uninterrupted for infinite durations without memory leaks crashing the application.

3. The Database (Storage & Persistence)

We implemented an embedded, serverless relational database system to persistently store all collected data without requiring users to configure external database installations.

- **Engine: SQLite**.
- **Transaction Management:** Driven via Rust bindings (`rusqlite`). Allows multi-batch ingestion scaling (e.g., the recent robust "Offline USB Upload" workflow) utilizing ACID-compliant transactional SQL `BEGIN` and `COMMIT` queries.
- **Location Schema:** Keeps an embedded `.sqlite` local database file tied securely inside the application context isolated for offline access globally.

4. Summary of Tech Choice Benefits

- **Blazingly Fast:** Using Tauri + Rust rather than Electron removes NodeJS background overhead, slashing memory consumption and resulting in a tiny application bundle (<10 MB).
- **Responsive Visuals:** Opting for Vanilla JS and DOM navigation means near-instant rendering speeds while providing granular chart functionality.
- **Highly Secure:** No arbitrary system access is permitted to the HTML side. All external reads/writes go specifically through authorized Rust endpoints (`invoke_handlers`).

Generated securely via antigravity.